

Complete ROM and RAG Pipeline Specification: Stage 1 → Stage 2 → Stage 3 → Final ROM

1. Executive Architectural Overview

The Record of Meeting (ROM) engine processes high-fidelity meeting transcripts, video optical character recognition (OCR) contexts, and background corporate documents through a **three-stage refinement and Retrieval-Augmented Generation (RAG) pipeline**. The architecture converts fragmented, conversational speech into a structured, agenda-aligned, and verified executive document.

2. Stage 1: Windowed Discussion Point Extraction

Implementation Files:

- `backend/services/rom_service.py:388` (`extract_discussion_points`)
- `backend/services/video_processing_service.py` (`get_overlapping_ocr_blocks`)
- `backend/routers/rom_router.py` (`Stage1Request`)

2.1 2-Minute Sliding Window Partitioning

Conversational transcripts are divided into temporal windows:

- **Default Window Size:** 2.0 minutes (120.0 seconds, configurable via `transcript_window_minutes` between 0.5 and 30.0 min).
- **Partitioning Rule:** As segments arrive, once $(\text{segment.start} - \text{window.start}) \geq 120.0\text{s}$, a new window is created.
- **Window Text Construction:** Transcript segments are formatted with timestamps and resolved speaker labels (preventing raw diarization IDs like `SPEAKER_00` from reaching the LLM):

```
[0.0-14.2] Alice: Let's review the cloud infrastructure budget for Q3.  
[14.5-28.0] Bob: Our AWS spend increased by 18% due to vector database indexing.
```

2.2 Video OCR Context Overlap Extraction

For recorded screen shares or video meetings, visual presentations (slides, dashboards, code) contain context absent from spoken dialogue.

- The engine calls `get_overlapping_ocr_blocks(video_transcript, w_start, w_end)`.
- OCR text blocks whose temporal intervals overlap the window `[w_start, w_end]` are extracted:

```
[01:15 -> 02:30]  
Slide 4: AWS Cost Optimization Breakdown  
- OpenSearch cluster: $4,200/mo  
- GPU Inference instances: $8,500/mo
```

2.3 Context Ingestion

Each window is given two additional contextual blocks:

1. **Previous Window Transcript:** Provides conversational continuity across turn boundaries.
2. **Video OCR Context:** Directly grounds technical terms, numbers, and diagram text.

2.4 LLM Processing Paths

Stage 1 supports two modes of execution:

Mode	Architecture	Description
Embedded (Default)	Single LLM Call per Window	Generates points with action items woven into the text. Derives <code>action_owner</code> directly.
Split-Call (<code>separate_action_extraction</code>)	Two Parallel LLM Calls	Call 1: Extracts pure discussion points. Call 2: Evaluates a 2× window (current + next window, 240 s) to extract explicit action items, mapping them back to discussion points via keyword overlap.

2.5 Stage 1 Output Schema

```
{
  "id": "c1f7a2d4-7b91-4c6e-821f-825d19a2e310",
  "window_index": 0,
  "timeline_start": 0.0,
  "timeline_end": 120.0,
  "discussion_point": "Cloud infrastructure expenditure rose by 18% during Q3, primarily driven by vector store ingesti",
  "speakers": ["Alice", "Bob"],
  "technical_terms": ["AWS", "vector store", "indexing", "OpenSearch"],
  "dates": ["Q3"],
  "numbers": ["18%"],
  "references": [],
  "action_items": [
    {
      "task": "Audit GPU instance utilization on AWS",
      "owner": "Bob",
      "deadline": "Friday"
    }
  ],
  "action_owner": "Bob",
  "raw_transcript_text": "[0.0-14.2] Alice: ...",
  "video_transcript_context": "[01:15 -> 02:30]\nSlide 4: AWS Cost..."
}
```

2.6 Single-Window Correction & Regeneration

Users can edit feedback on a specific window via `POST /rom/{recording_id}/stage1/rerun-window` :

- Takes `window_index` , `user_feedback` , and original output.
- LLM re-extracts points applying the user's specific instructions (`provider.extract_rom_discussion_points_with_feedback`).

3. Stage 2: Contextual Refinement, Validation & Deduplication

Implementation File: `backend/services/rom_service.py:1366` (`enhance_discussion_points`)

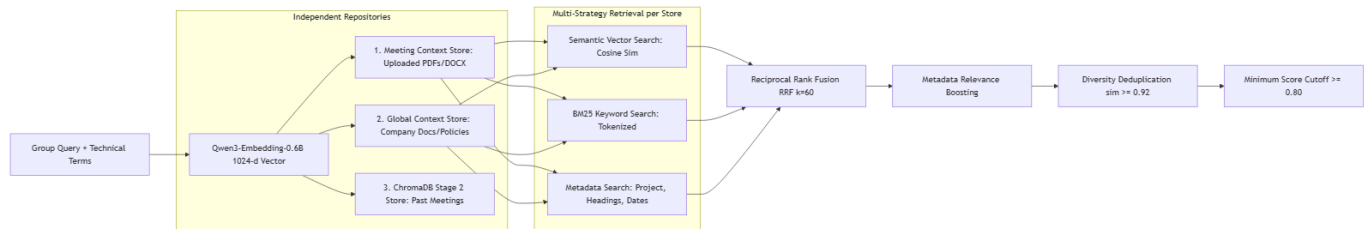
Stage 2 elevates granular 2-minute dialogue points into polished executive summaries, incorporates relevant external background documents, verifies claims against the transcript, and removes redundancy.

3.1 Point Grouping

- **Windowed Grouping (Default):** Discussion points from 5 consecutive Stage-1 windows (~ 10 minutes of meeting time) are batched (`discussion_window_size = 5`).
- **All-Together Mode (`process_all_together = True`):** All Stage-1 discussion points are processed in a single global context pass.

3.2 Hybrid RAG Retrieval (Vector + Keyword + Metadata)

For each group, a composite search query is formed from all discussion points and technical terms. Retrieval executes concurrently across three isolated vector repositories:



1. Retrieval Strategies & Reciprocal Rank Fusion (RRF)

Results from semantic vector search, BM25 token matching, and metadata filtering are fused:

$$\text{RRF_Score}(d) = \sum_{m \in \{\text{semantic}, \text{bm25}, \text{meta}\}} \frac{1}{60 + \text{rank}_m(d)}$$

2. Metadata Relevance Boosting

Fused candidates receive additive boosts:

- **Project Name Match:** +0.10 if matching entities in metadata.
- **Technical Entity Overlap:** +0.05 per match (up to +0.15).
- **Section / Heading Match:** +0.08 for chapter or topic overlap.
- **Document Title Match:** +0.10 if document name matches query terms.
- **Date Match:** +0.05.

3. Diversity Deduplication

Redundant context chunks (cosine similarity ≥ 0.92) are pruned, ensuring the LLM receives diverse, non-repetitive source material.

4. Score Filtering

Retrieved chunks with similarity < 0.80 (`min_similarity_threshold`) are excluded.

3.3 Context-Enriched LLM Enhancement

The LLM receives:

1. The group of discussion points.
2. Verified meeting context snippets.
3. Global corporate context snippets.
4. Past meeting historical decisions (from previous Stage 2 points).
5. Style guidelines (if reference example points are provided).

Task: Synthesizes and clarifies points, standardizes terminology, resolves acronyms, and generates `polished_text` while strictly maintaining factual bounds.

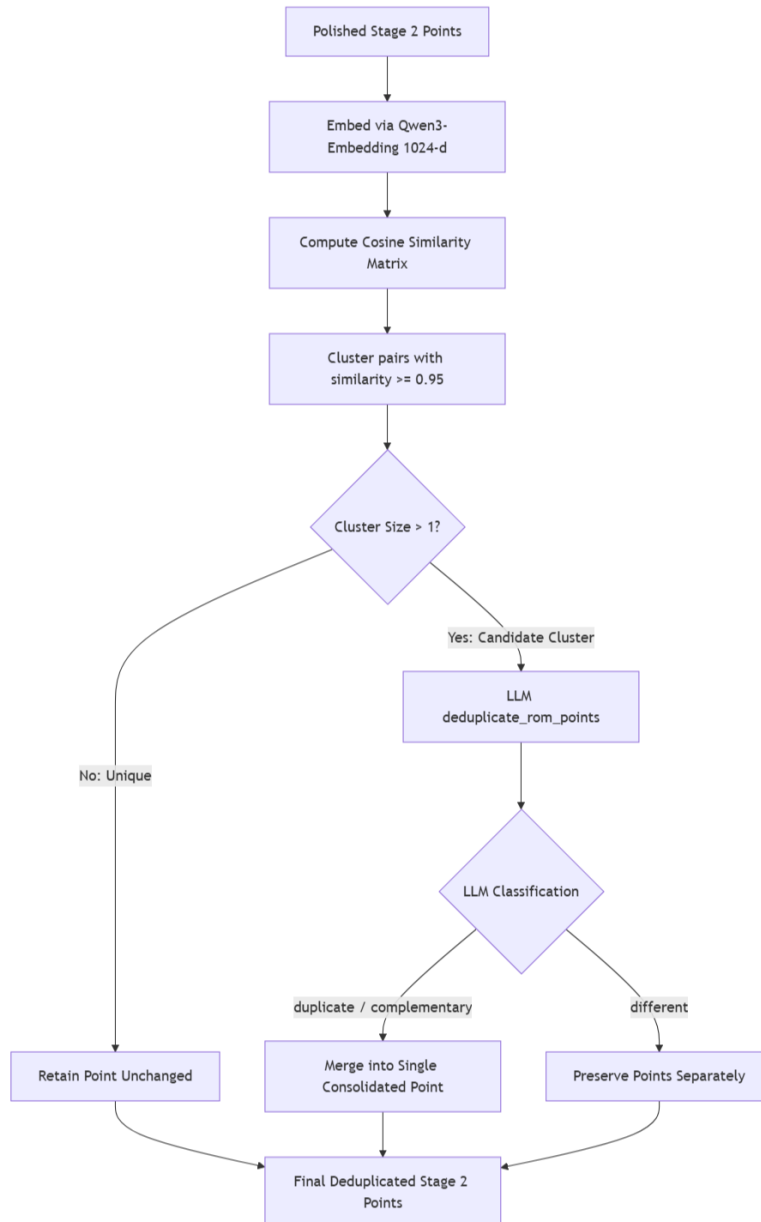
3.4 Factuality Validation & Correction Pass

To eliminate hallucinations introduced during enrichment:

1. `validate_stage2_point(pt_text, validation_transcript_text)` compares the enhanced text against the verbatim transcript.
2. If invalid, `correct_invalid_stage2_points()` re-aligns the point to match the transcript while retaining resolved terminology.

3.5 Semantic Deduplication Pass (Similarity ≥ 0.95 + LLM)

Discussions often revisit topics across different parts of a meeting. To deduplicate without losing nuance:



1. **Embedding Matrix:** Encodes all polished points using `Qwen3-Embedding-0.6B`.
2. **Cosine Clustering:** Identifies clusters where similarity ≥ 0.95 .
3. **LLM Verification (`provider.deduplicate_rom_points`):**
 - The LLM classifies the cluster as:
 - `"duplicate"` : Points describe the same update $\rightarrow$ merged.
 - `"complementary"` : Points describe different parts of the same topic $\rightarrow$ synthesized.
 - `"different"` : Points address distinct topics despite similar wording $\rightarrow$ kept separate.

- ▀ `different` : Points address distinct topics despite similar wording → kept separate.

4. Metadata Merging:

- `original_point_ids` : Combines all source Stage 1 IDs.
- `timeline_start` : $\min(\text{start})$.
- `timeline_end` : $\max(\text{end})$.
- `speakers` : Union of all active participants.
- `action_owner` : Inherited from the primary action item.

3.6 Cross-Meeting ChromaDB Indexing

Every finalized Stage 2 point is automatically indexed into ChromaDB (`stage2_points_<user_id>`):

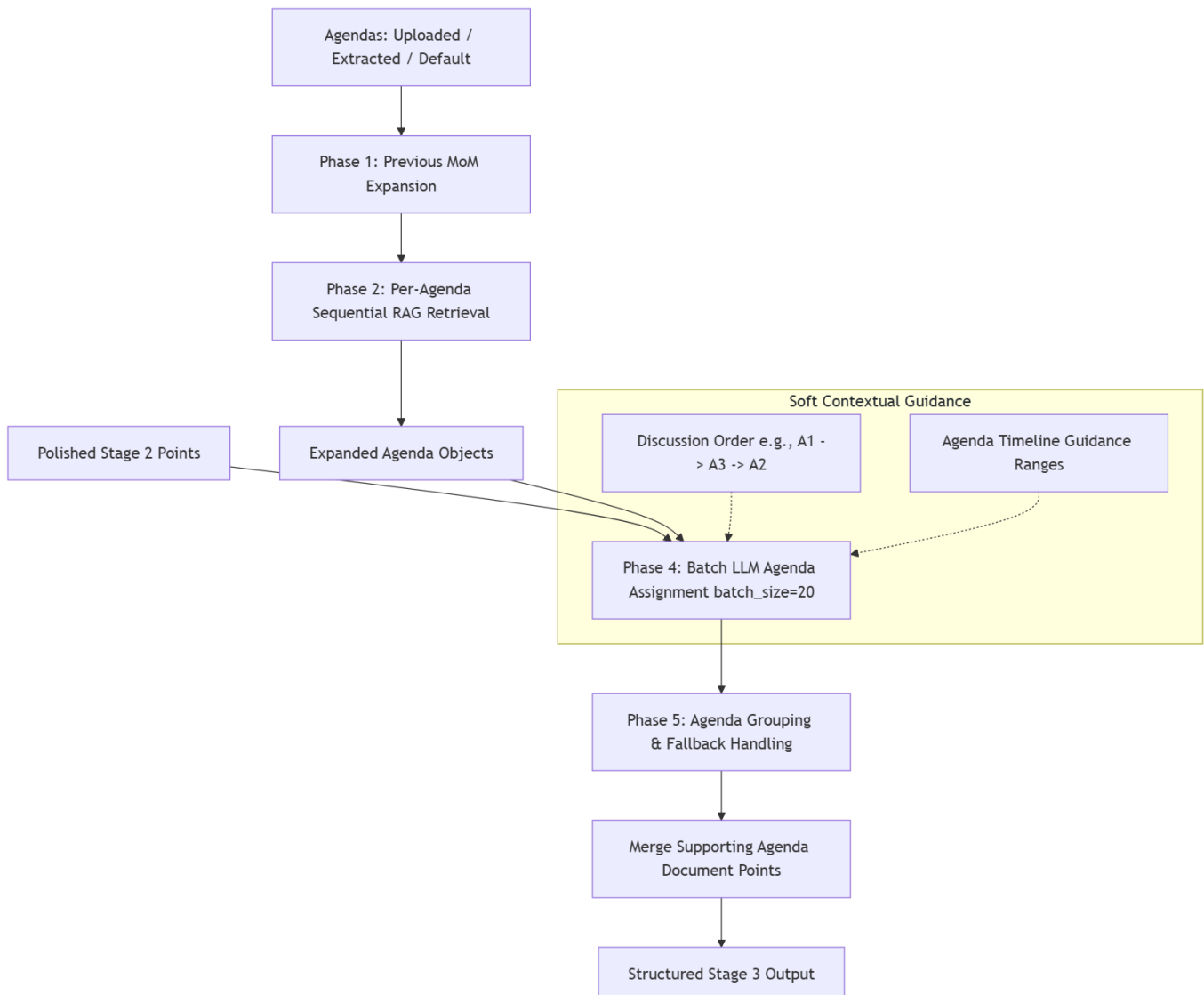
- Enables subsequent meetings to query past decisions, open items, and continuity context.

4. Stage 3: Agenda Matching and Topic Alignment

Implementation File: `backend/services/rom_service.py:2736` (`generate_agendas_and_map`)

Stage 3 aligns polished discussion points to the meeting's formal agenda items.

4.1 5-Phase Processing Architecture



4.2 Phase Details

Phase 1: Previous MoM Expansion

If previous meeting Minutes of Meeting documents are uploaded, `expand_agendas_with_previous_mom()` scans them to determine if an agenda item is a continuation of previous unfinished business, extracting background context.

Phase 2: Per-Agenda RAG Context Expansion

For each agenda item, the engine builds an expansion query:

Query = Title + Description + Keywords + Related Concepts + Expected Themes

RAG searches retrieve supporting meeting documents and corporate policies, compiling an `ExpandedAgenda` object.

Phase 3 & 4: Batch LLM Agenda Assignment

- To handle long meetings without token overflow, discussion points are chunked into batches of 20 (`batch_size = 20`).
- The LLM maps each point to its most relevant `agenda_id` .
- **Soft Guidance Inputs:**
 - `discussion_order` : Informs the LLM of the actual order topics were addressed (e.g., `A1 -> A3 -> A2`).
 - `agenda_timeline` : Approximate time boundaries for each agenda item.
- **Confidence Scoring:** Assigns `"high"` , `"medium"` , or `"probable"` .
- **Fallback Rule:** If the LLM returns an invalid or unrecognized ID, the point is mapped to the default agenda (`A1`) and flagged with `is_probable = True` and `confidence = "probable"` .

Phase 5: Agenda Grouping & Supporting Document Ingestion

- Points are grouped under their assigned agendas.
- **Supporting Agenda Document Points:** If documents were attached directly to an agenda item (e.g., a project status brief), factual points are extracted (`extract_agenda_document_points`) and appended with `speaker: "From Document"` and `is_doc_point: True` .

5. Final ROM Compilation and Minutes of Meeting (MoM)

Implementation Files:

- `backend/services/rom_service.py:3686` (`generate_mom_from_enhanced_rom`)
- `backend/routers/rom_router.py:1473` (`generate_final_rom_from_agendas`)
- `backend/services/speaker_sync.py` (`apply_speaker_mappings_to_final_rom`)

5.1 Skipped Agenda Reconciliation

If specific agendas were not discussed during the meeting, users can mark them as skipped:

- `skipped_agendas = {"A2": {"note": "Forwarded to next meeting due to time constraints"}}` .
- Skipped agendas are preserved in their original numerical position in the final document, marked with `skipped: True` , while discussion point mapping skips them.

5.2 Action Point Resolution & Owner Attribution

Action owners are assigned via a 5-step fallback hierarchy:

1. **Explicit LLM Assignee:** Owner identified directly in the text.

- 2. **Stage 1 Extracted Owner:** Preserved from window extraction.
- 3. **Primary Turn Speaker:** Assigned to the speaker who proposed the commitment.
- 4. **Participant Name Matching:** Participant list searched for names appearing in the task description.
- 5. **Fallback:** Assigned to "Unassigned" .

5.3 Global Speaker Synchronization (`speaker_sync.py`)

If a speaker was renamed after transcription (e.g., `Speaker 1` → `Alice`), `apply_speaker_mappings_to_final_rom()` recursively updates all speaker fields, participants lists, and action item owners throughout the ROM and MoM data structures.

5.4 Formal MoM Generation & Constraints

The executive MoM generates a standardized record:

Section	Generation Logic & Strict Constraints
Title	Inferred from meeting topics rather than using raw filenames (e.g., "Q3 Infrastructure & Budget Review").
Introduction	Strict Constraint: Exactly one single paragraph, 5 to 7 sentences. Summarizes the meeting purpose, key participants, and core themes without filler.
Points Discussed	Direct projection of Stage 2 polished points grouped under Stage 3 agendas.
Action Items	Structured table containing <code>task</code> , <code>owner</code> , <code>deadline</code> (date or "ASAP"), and <code>status</code> ("open").
Conclusion	Strict Constraint: Exactly one single paragraph, 5 to 7 sentences. Summarizes consensus, agreed milestones, open risks, and next steps.

5.5 Document Export Versions

The Final ROM compiles into three selectable versions:

- **Long Version:** Complete, unabridged record including all contextual justifications, technical term glossaries, timeline timestamps, and supporting document citations.
- **Medium Version:** Executive summary preserving all discussion points and action items with timeline metadata stripped.
- **Short Version:** High-level overview containing Title, Introduction, Agendas, Action Items table, and Conclusion.

Export formats supported: Microsoft Word (`.docx`), PDF, and JSON API response.

6. Pipeline Component and Technology Summary

Stage	Component	Technology / Library	Parameters & Thresholds	Purpose
Stage 1	Sliding Window Splitter	Python / NumPy	<code>window_minutes = 2.0</code>	Slices conversational transcripts into manageable temporal windows.
Stage 1	OCR Overlap Locator	Python / Video Service	Temporal intersection > 0.0 s	Locates visual slide/presentation text corresponding to the window.
Stage 1	Window LLM Extractor	Local Qwen / Ollama / Cloud	Concurrency: 2 (max 5)	Generates initial discussion points and action items from transcripts.
Stage 2	Text Embedding Engine	Qwen3-Embedding-0.6B	Dimension: 1024 float32	Converts discussion points into semantic vectors for RAG and clustering.
Stage 2	Context Vector Stores	ChromaDB / FAISS	Metric: Inner Product / Cosine	Stores embedded chunks for meeting documents and global company data.
Stage 2	Keyword Search Engine	Custom BM25 Index	Tokenized inverse document freq	Keyword retrieval complementary to semantic vector search.
Stage 2	Rank Fusion Engine	Reciprocal Rank Fusion	Constant $k = 60$	Merges vector, BM25, and metadata search rankings.
Stage 2	Diversity Filter	Vector Cosine Similarity	Threshold: 0.92	Prunes redundant background context chunks before prompt construction.
Stage 2	Relevance Cutoff	Vector Cosine Similarity	Threshold: 0.80	Excludes weakly related background document chunks.
Stage 2	Validation & Correction	LLM Inference	Transcript plain text prompt	Verifies that enhanced points do not hallucinate beyond the transcript.
Stage 2	Semantic Deduplication	Vector Cosine Similarity	Threshold: 0.95	Clusters near-duplicate points across windows for LLM consolidation.
Stage 2	Cross-Meeting Store	ChromaDB (<code>stage2_points</code>)	User-isolated collection	Indexes polished points to provide historical context for future meetings.
Stage 3	MoM Continuity Expander	LLM Inference	Character limit: 20 000	Detects whether agenda items continue topics from previous meeting MoMs.
Stage 3	Batch Agenda Mapper	LLM Inference	<code>batch_size = 20</code>	Assigns discussion points to agenda items using soft order and timeline evidence

				guidance.
Final ROM	Overview Generator	LLM Inference	Prompt constraints: 5–7 sentences	Generates executive single-paragraph Introduction and Conclusion.
Final ROM	Speaker Synchronizer	Python (<code>speaker_sync.py</code>)	Deep JSON recursive mapping	Propagates speaker renames across all nested ROM and MoM fields.